

REVERSE ENGINEERING // MASTERY COURSE

PHASE 0

Foundation

Neev majboot karo — number systems, memory, C basics aur compilation.

```
re@learning: ~/phase-0

$ ./start_learning.sh --phase 0
# Hinglish mein, simple examples ke saath – ek-ek step
$ status: READY ✓
```

 Detailed Theory

 Code + Line-by-line

 Interview Q&A

IN Hinglish

Reverse Engineering Mastery // Self-Study Roadmap // Phase 0 Handbook

Phase 0 — Foundation (Neev Majboot Karo)

Goal: Reverse Engineering seekhne se pehle computer andar se kaise kaam karta hai — ye samajhna. Ye neev hai. Agar ye kachhi rahi, toh aage sab kuch upar se nikal jaayega. Time: **2-3 hafte**.

0.0 — Sabse Pehle: Ye Phase Kyun?

Socho tum ek doctor banna chahte ho. Kya seedha operation seekhoge? Nahi. Pehle body ke parts (anatomy) padhoge. Waise hi RE mein, pehle "computer ki anatomy" samajhni padegi — numbers kaise store hote hain, memory kaise kaam karti hai, program kaise banta hai.

Is phase mein 4 cheezein seekhenge: 1. **Number Systems** (Binary, Hex, Decimal) 2. **Memory & Data** (RAM, bits, bytes, endianness) 3. **C Programming basics** (kyunki zyada tar programs C mein bane hote hain) 4. **Compilation** (source code se `.exe` kaise banta hai)

0.1 — Number Systems (Binary, Hex, Decimal)

Computer sirf 0 aur 1 samajhta hai

Humari duniya **decimal** (0-9, 10 digits) use karti hai. Lekin computer ke andar sirf electricity hai — ya toh current hai (**1**) ya nahi hai (**0**). Isi ko **Binary** kehte hain.

- **Binary (Base 2):** sirf `0` aur `1`
- **Decimal (Base 10):** `0` se `9` (hum log)
- **Hexadecimal / Hex (Base 16):** `0-9` phir `A, B, C, D, E, F`

Hex kyun? (RE ka favourite)

Binary bahut lambi hoti hai. `11111111` likhna mushkil. Isliye RE mein hum **Hex** use karte hain — chhoti aur aasan. Har hex digit = 4 binary digits.

```
Binary:  1111 1111
Hex:     F    F    = 0xFF
```

`0x` prefix ka matlab hai "ye hex number hai". Jaise `0x41`, `0xFF`. RE tools mein sab jagah `0x`

dikhega.

Conversion Table (yaad rakhne layak)

Decimal	Binary	Hex
0	0000	0x0
5	0101	0x5
10	1010	0xA
15	1111	0xF
16	10000	0x10
65	1000001	0x41
255	11111111	0xFF

Real Example — Letter 'A'

Computer mein har letter ek number hota hai (ASCII code):

```
Letter 'A' = Decimal 65 = Binary 01000001 = Hex 0x41
Letter 'B' = Decimal 66 = Hex 0x42
Letter 'a' = Decimal 97 = Hex 0x61
```

RE karte waqt jab tumhe memory mein **41 42 43** dikhega, tum turant samajh jaoge ki wo **"ABC"** hai. Ye skill baar-baar kaam aayegi!

Practice

- Decimal 200 ko hex mein convert karo (answer: **0xC8**)
- Hex **0x2A** ko decimal mein (answer: **42**)
- Windows calculator ko "Programmer mode" mein rakh ke practice karo — ye instant convert karta hai.

0.2 — Memory, Bits & Bytes

Bits aur Bytes

- **Bit** = ek 0 ya 1 (sabse chhoti unit)
- **Byte** = 8 bits (11111111) — ek byte se 0 se 255 tak number ban sakta hai
- Aage: 1 KB = 1024 bytes, 1 MB = 1024 KB...

```
1 byte = 8 bits = 2 hex digits
Example: 0xFF = 11111111 = 255 (ek byte ki max value)
```

Memory (RAM) — Ghar ke address jaisa

RAM ko ek badi building socho jismein crore-o flats hain. Har flat ka ek **address** hai. Computer har data ko kisi address pe rakhta hai.

```
Address      Value (byte)
0x1000 →     0x41 ('A')
0x1001 →     0x42 ('B')
0x1002 →     0x43 ('C')
```

RE mein hum baar-baar "is address pe kya value hai?" ye dekhte hain. Debugger memory ko exactly aise hi dikhata hai.

Endianness (★ Ye confuse karta hai — dhyan se)

Jab ek number ek byte se bada ho (jaise 4-byte number), toh usko memory mein kis order mein rakhein? Do tarike hain:

- **Little Endian:** chhota (least significant) byte pehle. **Intel/AMD (x86/x64) yahi use karte hain.**
- **Big Endian:** bada byte pehle (network protocols mein common).

Example: Number 0x12345678 ko memory mein store karna:

```
Little Endian (Intel PC):  78 56 34 12  <- ULTA! bytes reversed
Big Endian:                12 34 56 78  <- seedha
```

Beginners yahin confuse hote hain. Jab debugger mein `78 56 34 12` dikhe, actual number `0x12345678` hai. Bas byte order ulta hai. Ye yaad rakho.

0.3 — C Programming Basics (RE Ki Jaan)

Zyada tar software (Windows, games, malware) **C/C++** mein bane hote hain. Agar tum C samajhte ho, toh decompiled code (Ghidra ka output) padhna aasan ho jaayega — kyunki wo C jaisa hi dikhta hai.

Ek simple C program

```
#include <stdio.h>           // input/output ke functions laane ke liye (printf)

int main() {                 // main = program yahi se shuru hota hai
    int age = 25;            // 'age' naam ka box banaya, usme 25 rakha
    printf("Age: %d", age);  // screen pe "Age: 25" chhaapo (%d = number)
    return 0;               // program theek se khatam (0 = success)
}
```

Line by line samjho: - `#include <stdio.h>` — ye "library" laata hai jisme `printf` jaisa function hota hai. Isse pehle likhna padta hai. - `int main()` — har C program `main` se chalu hota hai. `int` matlab ye ek number return karega. - `int age = 25;` — `int` = integer (poora number). `age` variable ka naam. `= 25` uski value. - `printf("Age: %d", age);` — screen pe print. `%d` ki jagah `age` ki value (25) aa jaayegi. - `return 0;` — program ko batana ki sab theek raha (0 = no error).

Zaroori Concepts

1. Variables & Data Types

```
int    count = 10;          // poora number (integer)
char    letter = 'A';        // ek character (asal mein number 65)
float  price = 9.99;         // decimal number
```

2. Pointers (★ ★ Sabse important, RE ka dil)

Pointer ek variable hai jo kisi doosre variable ka **address** rakhta hai (value nahi, ghar ka pata).

```
int x = 5; // x mein 5 hai (maano address 0x1000 pe)
int *p = &x; // p mein x ka ADDRESS hai (0x1000), value nahi
printf("%d", *p); // *p matlab "us address pe jo value hai" = 5
```

Line by line: - `int x = 5;` — normal variable, value 5. - `int *p = &x;` — `*` batata hai ye pointer hai. `&x` matlab "x ka address". Toh `p` ke andar x ka address hai. - `*p` — `*` yahan "dereference" hai = "is address pe jaake value nikaalo" = 5.

Pointers samajh gaye toh RE ka 40% clear. Kyunki assembly mein sab kuch addresses se hota hai.

`&` = address lena, `*` = us address pe jaana.

3. Functions, Loops, If-else

```
int add(int a, int b) { // 'add' function, do numbers leta hai
    return a + b; // dono ko jodke wapas bhejta hai
}

for (int i = 0; i < 5; i++) { // loop: i=0 se 4 tak, 5 baar chalega
    printf("%d ", i); // print: 0 1 2 3 4
}

if (age >= 18) { // condition check
    printf("Adult");
} else {
    printf("Minor");
}
```

4. Arrays & Strings

```
int nums[3] = {10, 20, 30}; // 3 numbers ka array
char name[] = "Amit"; // string = characters ka array (A,m,i,t,\0)
```

String ke end mein ek chhupa `\0` (null byte) hota hai jo batata hai "string yahan khatam". RE mein ye dikhega.

Practice

- Ek `hello.c` likho, compile karo, chalao.

- Pointer wala program likho aur samjho `&` aur `*` kaise kaam karte hain.

0.4 — Compilation: Source Code Se .exe Tak

Ye samajhna **bahut zaroori** hai, kyunki RE isi process ka **ulta** hai.

Forward Process (normal programming)

```
hello.c      →  Compiler  →  hello.exe
(hum likhte hain)    (gcc/msvc)    (computer chalata hai)
```

Detail mein:

Source Code (.c)

↓ (Preprocessor — #include waale kaam)

Expanded Code

↓ (Compiler)

Assembly Code (.asm)

↓ (Assembler)

Machine Code / Object (.obj)

↓ (Linker — libraries jodta hai)

Executable (.exe)

Reverse Process (RE — hum ye karte hain)

```
hello.exe  →  Disassembler  →  Assembly  →  (hum samajhte hain logic)
              (Ghidra/IDA)              →  Decompiler → C-jaisa code
```

Yaad rakho: compile karte waqt bahut si information kho jaati hai — variable ke naam (`age` , `price`), comments, sab gayab. Isliye RE mushkil hai — hume ye khoyi hui info wapas guess karni padti hai.

Khud Karke Dekho (Best Practice)

`gcc -S` command source ko assembly mein badalti hai (compile nahi karti, sirf assembly dikhati hai):

```
gcc -S hello.c -o hello.s    # hello.s mein assembly milegi
gcc hello.c -o hello.exe     # normal compile
```

Sabse best tool: godbolt.org - Left side mein C likho, right side mein turant assembly dikhegi. - Ye Phase 1 (Assembly) ke liye bhi bahut kaam aayega. Abhi se use karna shuru karo.



Phase 0 — Hands-On Checklist

- [] Windows Calculator "Programmer mode" mein hex ↔ decimal convert kiya
- [] `'A'` = 65 = 0x41 samajh liya (ASCII)
- [] Little vs Big endian ka farak samajh liya (`0x12345678` → `78 56 34 12`)
- [] Ek `hello.c` likha, compile kiya, chalaya
- [] Pointer wala program likha (`&x` aur `*p`)
- [] godbolt.org pe C likh ke assembly dekhi



Key Takeaways (Ek Line Mein)

1. Computer sirf **binary** (0/1) samajhta hai; hum **hex** (0x..) use karte hain kyunki chhoti hoti hai.
2. **1 byte = 8 bits = 2 hex digits**; ek byte 0-255 tak.
3. Har data RAM mein ek **address** pe hota hai.
4. **Little endian** = bytes ulte store hote hain (Intel PC). `0x12345678` → `78 56 34 12` .
5. **Pointer** = address rakhne wala variable. `&` = address lo, `*` = address pe jaao. (RE ka dil)
6. Compile = source → exe (info kho jaati hai). RE = exe → samajhna (info wapas guess karna).

✅ **Phase 0 Complete Jab:** Tum hex-decimal convert kar sako, ek chhota C program (pointer ke saath) likh-compile kar sako, aur memory/endianess ka clear idea ho. Ab Phase 1 (Assembly) ke liye ready ho! 🚀

Phase 0 — Interview Questions & Answers (Hinglish)

Ye important theory questions hain jo interview mein Phase 0 (Foundation) se puche ja sakte hain. Har jawaab simple Hinglish mein — jaise tum interviewer ko bologe.

Q1. Reverse Engineering kya hai?

Ans: Reverse Engineering ka matlab hai kisi cheez ko "ulta" samajhna — jab humein source code nahi pata, tab hum ek program (jaise `.exe`) ko todkar, andar dekhkar samajhte hain ki wo kaise kaam karta hai. Jaise ek band ghadi ko kholkar samajhna ki gears kaise chalte hain, waise hi software pe.

Q2. Computer sirf binary (0 aur 1) kyun samajhta hai?

Ans: Computer ke andar electronic circuits hote hain jinme sirf do states hoti hain — current on (**1**) ya off (**0**). Isiliye computer sab kuch binary mein store aur process karta hai. Hardware level pe sirf yahi do cheezein possible hain.

Q3. Hexadecimal kyun use karte hain, binary kyun nahi?

Ans: Binary bahut lambi hoti hai — jaise `11111111`. Hex chhoti aur padhne mein aasan hai — wahi cheez `0xFF` ho jaati hai. Ek hex digit exactly 4 binary digits (bits) ko represent karta hai, isliye conversion bhi seedha hai. RE tools memory aur values hex mein hi dikhte hain.

Q4. 1 byte mein kitni values aa sakti hain?

Ans: 1 byte = 8 bits. 8 bits se $2^8 = 256$ **alag values** ban sakti hain, matlab **0 se 255** tak. Hex mein ye `0x00` se `0xFF` tak hota hai.

Q5. Memory address kya hota hai?

Ans: RAM ek badi building jaisi hai jismein har "flat" ka ek unique number hota hai — usi ko memory address kehte hain. Computer har data (variable, code) ko kisi address pe rakhta hai, taaki baad mein use dhoond sake. RE mein hum baar-baar dekhte hain ki kisi address pe kya value hai.

Q6. Endianness kya hoti hai? Little aur Big endian mein kya farak hai?

Ans: Endianness batati hai ki ek multi-byte number memory mein kis order mein store hoga. - **Little Endian:** sabse chhota (least significant) byte pehle store hota hai. Intel/AMD (x86/x64) yahi use karte hain. - **Big Endian:** sabse bada byte pehle. Network protocols mein common.

Example: `0x12345678` little endian mein memory mein `78 56 34 12` (ulta) dikhega, big endian mein `12 34 56 78`.

Q7. Pointer kya hota hai? Isse RE mein kya importance hai?

Ans: Pointer ek variable hai jo kisi doosre variable ki **value nahi, balki address** rakhta hai. `&x` se x ka address milta hai, aur `*p` se us address pe rakhi value milti hai. RE mein pointer bahut important hai kyunki assembly level pe zyada tar kaam addresses ke through hota hai — data ko address se access kiya jaata hai.

Q8. ASCII kya hai? 'A' ki value kya hoti hai?

Ans: ASCII ek standard hai jo har character ko ek number deta hai, taaki computer text ko store kar sake. Jaise `'A'` = 65 (hex `0x41`), `'a'` = 97 (`0x61`), `'\0'` = 48. RE mein jab memory mein `41 42 43` dikhta hai, wo actually `"ABC"` hota hai.

Q9. Compilation process kya hai? Source code se .exe kaise banta hai?

Ans: Steps hote hain: 1. **Preprocessor** — `#include` jaise kaam handle karta hai. 2. **Compiler** — C code ko assembly mein badalta hai. 3. **Assembler** — assembly ko machine code (object file) mein. 4. **Linker** — libraries jodkar final `.exe` banata hai.

RE isi ka **ulta** hai — hum `.exe` se wapas samajhne ki koshish karte hain.

Q10. Compile karne pe kaunsi information kho jaati hai?

Ans: Compile hone pe variable ke naam (jaise `age`, `price`), function ke asli naam, comments, aur data types ki clear info kho jaati hai. Machine code mein sirf raw instructions bachti hain. Isliye RE mushkil hota hai — hume ye khoyi hui information wapas guess karni padti hai.

Q11. Machine code aur Assembly mein kya farak hai?

Ans: **Machine code** pure numbers (0s aur 1s / bytes) hote hain jo CPU seedha chalata hai. **Assembly** usi machine code ka human-readable version hai — jaise `mov eax, 5`. Har assembly instruction ek machine code se map hoti hai. Assembly humein padhne mein aasan hai.

Q12. Stack aur Heap mein basic farak kya hai?

Ans: Dono memory ke hisse hain jahan program data rakhta hai. - **Stack:** automatic, fast, function ke local variables aur call info yahan aati hai. Order fixed hota hai (LIFO — Last In First Out). - **Heap:** manual/dynamic memory, jise program chalte-chalte maangta hai (jaise C mein `malloc`). Zyada flexible but manage karna padta hai.

Q13. Bit aur Byte mein farak?

Ans: **Bit** sabse chhoti unit hai — ek `0` ya `1`. **Byte** = 8 bits ka group. Ek byte se 0-255 tak number ban sakta hai. Data usually bytes mein measure hota hai (KB, MB, GB).

Q14. RE seekhne ke liye C language kyun zaroori hai?

Ans: Kyunki zyada tar software C/C++ mein bane hote hain, aur decompiler (jaise Ghidra) ka output C jaisa dikhta hai. Agar tum C samajhte ho — especially pointers, arrays, functions — toh decompiled code padhna bahut aasan ho jaata hai.

Phase 0 — Interview Questions & Answers (English)

The same important theory questions from Phase 0 (Foundation), answered in clear English. Use this to phrase your exact answer in a formal interview.

Q1. What is Reverse Engineering?

Ans: Reverse Engineering is the process of understanding how something works by taking it apart — without having its source code. In software, we take a compiled program (like an `.exe`) and analyze it to figure out its internal logic. It's like opening a closed watch to understand how the gears drive it, but applied to software.

Q2. Why does a computer only understand binary (0s and 1s)?

Ans: A computer's electronic circuits have only two stable states — current on (**1**) or off (**0**). Because hardware can reliably represent only these two states, everything is stored and processed in binary.

Q3. Why do we use hexadecimal instead of binary?

Ans: Binary is long and hard to read (e.g., `11111111`). Hexadecimal is compact and readable — the same value becomes `0xFF`. Each hex digit maps exactly to 4 binary bits, so conversion is straightforward. RE tools display memory and values in hex.

Q4. How many values can a single byte hold?

Ans: One byte = 8 bits. With 8 bits you get $2^8 = 256$ distinct values, i.e. 0 to 255 (hex `0x00` to `0xFF`).

Q5. What is a memory address?

Ans: RAM is like a large building where every "flat" has a unique number — that number is the memory address. The computer stores every piece of data (variables, code) at some address so it can locate it later. In RE we frequently inspect what value sits at a given address.

Q6. What is endianness? Difference between little and big endian?

Ans: Endianness defines the byte order in which a multi-byte number is stored in memory. - **Little Endian:** the least significant byte is stored first. Used by Intel/AMD (x86/x64). - **Big Endian:** the most significant byte is stored first. Common in network protocols.

Example: `0x12345678` is stored as `78 56 34 12` in little endian, and `12 34 56 78` in big endian.

Q7. What is a pointer? Why is it important in RE?

Ans: A pointer is a variable that holds the **address** of another variable rather than its value. `&x` gives the address of `x`, and `*p` gives the value stored at that address. Pointers are crucial in RE because at the assembly level most data access happens through addresses.

Q8. What is ASCII? What is the value of 'A'?

Ans: ASCII is a standard that assigns a number to each character so computers can store text. For example, `'A'` = 65 (hex `0x41`), `'a'` = 97 (`0x61`). In RE, bytes like `41 42 43` in memory actually represent the string `"ABC"` .

Q9. What is the compilation process — how does source code become an .exe?

Ans: The steps are: 1. **Preprocessor** — handles directives like `#include` . 2. **Compiler** — translates C code into assembly. 3. **Assembler** — converts assembly into machine code (object files). 4. **Linker** — links libraries and produces the final `.exe` .

Reverse Engineering is the reverse of this — going from `.exe` back to understanding the logic.

Q10. What information is lost during compilation?

Ans: Variable names (like `age`, `price`), original function names, comments, and clear type information are lost. Only raw instructions remain in the machine code. This is why RE is hard — we must reconstruct that lost information by analysis.

Q11. What is the difference between machine code and assembly?

Ans: Machine code is pure numbers (bytes) that the CPU executes directly. **Assembly** is a human-readable representation of that machine code, e.g., `mov eax, 5`. Each assembly instruction maps to machine code, but assembly is far easier for humans to read.

Q12. What is the basic difference between the stack and the heap?

Ans: Both are regions of memory a program uses. - **Stack:** automatic and fast; stores local variables and function call information in LIFO (Last In, First Out) order. - **Heap:** dynamic memory requested at runtime (e.g., `malloc` in C). More flexible but must be managed manually.

Q13. What is the difference between a bit and a byte?

Ans: A **bit** is the smallest unit — a single `0` or `1`. A **byte** is a group of 8 bits and can represent values from 0 to 255. Data sizes are usually measured in bytes (KB, MB, GB).

Q14. Why is knowing C important for learning RE?

Ans: Most software is written in C/C++, and decompiler output (e.g., from Ghidra) resembles C. If you understand C — especially pointers, arrays, and functions — reading decompiled code becomes much easier.